Git – Basics Überblick

1. Was ist Git?

Git ist ein **Versionskontrollsystem**. Es speichert den kompletten Verlauf deines Projekts, sodass du jederzeit zu älteren Ständen zurückkehren und Änderungen nachvollziehen kannst.

- **Git** = das Tool auf deinem Rechner (dezentrale Versionsverwaltung)
- **GitHub** = Hosting-Plattform für Git-Repos (Code im Web, Zusammenarbeit)

2. Installation & Setup (einmalig)

```
# Prüfen, ob Git installiert ist
git --version

# Benutzerdaten setzen (wichtig für Commits!)
git config --global user.name "Dein Name"
git config --global user.email "deine@mail.com"

# Neue Repos starten direkt mit dem Branch "main"
git config --global init.defaultBranch main
```

> **Schulrechner: --global schlägt fehl?**
> `--global` braucht keine Admin-Rechte, schreibt aber in die Datei `.gitconfig` im Benutzerprofil. Ist das Profil gesperrt, erscheint z. B. `could not lock config file ... Permission denied`. Zwei Auswege:
> - **Nur für ein Repo:** im Projektordner dieselben Befehle ohne `--global` ausführen (`git config user.name "Dein Name"`). Den Branchnamen setzt dann `git init -b main`.
> - **Config-Datei auf ein beschreibbares Laufwerk legen** (einmalig, ohne Admin-Rechte): `setx GIT_CONFIG_GLOBAL "U:\Dokumente\.gitconfig"`, danach ein neues Terminal öffnen. Ab jetzt schreibt `--global` in diese Datei.

3. Ein Repo lokal anlegen

```
cd mein-projekt     # in den Projektordner wechseln
git init            # neues Git-Repo initialisieren
```

Es entsteht ein versteckter Ordner `.git/` – dort verwaltet Git alles.

Bevor du das erste Mal `git add .` ausführst: Lege die `.gitignore` an (nächster Abschnitt).

4. .gitignore – Dateien ausschließen

Bestimmte Dateien (Passwörter, Build-Artefakte, Abhängigkeiten) gehören **nicht** ins Repo. Lege dafür im Projektordner eine Datei `.gitignore` an, z. B.:

```
node_modules/
.env
*.log
__pycache__/
dist/
```

Was schon einmal committed wurde, bleibt im Verlauf – deshalb die `.gitignore` *vor* dem ersten Commit anlegen.

5. Der zentrale Workflow

Git arbeitet mit **vier Orten**:

```
Arbeitsordner ──► Staging ──► Lokales Repository ──► Remote (GitHub)
  (arbeiten)     git add        git commit              git push
```

Typischer Ablauf:

```
git add .                          # 1. Änderungen vormerken (stagen)
git commit -m "Login-Formular ergänzt"   # 2. mit Nachricht speichern
git push                           # 3. auf GitHub hochladen
```

Faustregel: `add` = vormerken, `commit` = speichern mit Kommentar, `push` = hochladen.

Zum Nachsehen:

```
git status                # welche Dateien sind geändert bzw. gestaged?
git diff                  # konkrete Änderungen (noch nicht gestaged)
git log --oneline         # kompakter Verlauf
```

Gute Commit-Nachrichten: kurz (ca. 50 Zeichen), im Imperativ oder Perfekt, und sie sagen, **was** sich geändert hat – nicht „Änderungen“ oder „asdf“. Ein Commit sollte eine logische Änderung enthalten.

6. Projekt auf GitHub bringen

Das klassische Problem: Du hast lokal ein Projekt, GitHub kennt es noch nicht. Zuerst muss ein **leeres** Repo auf GitHub entstehen (ohne README/License, sonst gibt es Konflikte).

Weg A – über die Website:

1. GitHub → + oben rechts → **New repository**
2. Namen vergeben, Public/Private wählen, **nichts** anhaken
3. Lokal verbinden und hochladen:

```
git remote add origin https://github.com/dein-user/mein-repo.git
git push -u origin main
```

Weg B – GitHub CLI (sofern installiert), alles in einem Schritt:

```
gh repo create mein-repo --private --source=. --push
```

Das `-u` merkt sich die Verbindung; danach reicht `git push`. Mit `git remote -v` siehst du die verknüpften Remotes.

7. Ein Repo von GitHub holen (clone, pull, fetch)

```
git clone https://github.com/user/repo.git   # erstes Mal: komplette Kopie
git pull                                     # später: Änderungen holen
```

`git pull` = `git fetch` + `git merge`. Wer vorsichtiger vorgehen will, trennt die Schritte:

```
git fetch                 # lädt Änderungen, verändert deinen Code NICHT
git diff origin/main      # in Ruhe ansehen, was neu ist
git merge origin/main     # dann zusammenführen
```

Wichtig: Klone das Repo **nie** als ZIP herunter – dann fehlt die Git-Verbindung!

8. Branches – parallele Entwicklung

Branches erlauben es, an Features zu arbeiten, ohne `main` zu gefährden.

```
git branch                      # alle Branches anzeigen (* = aktuell)
git switch -c feature-login     # neuen Branch erstellen und wechseln
git switch main                 # zurück zu main
git branch -d feature-login     # Branch löschen (nach dem Merge)
```

Ältere Anleitungen nutzen `git checkout -b feature-login` – das funktioniert weiterhin.

Typischer Ablauf:

```
git switch -c feature-x
# ... arbeiten, add, commit ...
git switch main
git merge feature-x
git branch -d feature-x
```

9. Merge und Merge-Konflikte

```
git switch main           # auf den Ziel-Branch wechseln
git merge feature-x       # anderen Branch einfließen lassen
```

Wenn es knallt: Haben beide Seiten dieselbe Zeile geändert, meldet Git `CONFLICT (content)`. Öffne die Datei und entscheide, was stehen bleiben soll:

```
<<<<<<< HEAD
deine Version (aktueller Branch)
=======
die andere Version (feature-x)
>>>>>>> feature-x
```

Marker entfernen, gewünschten Text stehen lassen, dann:

```
git add datei.txt
git commit                # Merge abschließen
```

10. Zusammenarbeit über GitHub (Pull Requests)

In Teams arbeitet niemand direkt auf `main`. Stattdessen:

```
git switch -c feature-x           # 1. eigenen Branch anlegen
# ... arbeiten, add, commit ...
git push -u origin feature-x      # 2. Branch auf GitHub hochladen
```

1. Auf GitHub erscheint der Button **Compare & pull request** → Pull Request (PR) eröffnen
2. Andere schauen drüber, kommentieren, geben frei; dann wird der PR gemergt
3. Lokal aufräumen:

```
git switch main
git pull                          # gemergten Stand holen
git branch -d feature-x           # alten Branch löschen
```

Fremde Projekte, in die du nicht direkt pushen darfst, bearbeitest du über einen **Fork** (eigene Kopie auf GitHub) und stellst von dort den Pull Request.

11. Rückgängig machen & Zwischenablage

```
git restore --staged datei.txt   # aus Staging entfernen (Änderung bleibt)
git restore datei.txt            # Änderung in der Datei verwerfen
git reset --soft HEAD~1          # letzten Commit zurücknehmen (Änderung bleibt)
```

Achtung: unwiderruflich
`git restore datei.txt` verwirft deine nicht committeten Änderungen **endgültig** – es gibt kein Zurück.

Kurz wechseln, ohne zu committen: `git stash` legt Änderungen in eine Zwischenablage.

```
git stash          # uncommittete Änderungen wegpacken
git switch anderer-branch   # ... woanders arbeiten, zurückwechseln ...
git stash pop      # Änderungen wieder zurückholen
```

Fortgeschritten
- `git commit --amend -m "Neue Nachricht"` ändert den letzten Commit (nur vor dem Push!)
- `git reset --hard HEAD~1` wirft den letzten Commit *samt* Änderungen weg
- `git branch -D name` löscht einen Branch auch ohne Merge
- `git reflog` zeigt frühere Zustände und rettet oft versehentlich Gelöschtes

12. SSH-Key einrichten (Anmeldung bei GitHub)

Statt Passwort oder Token meldest du dich mit einem **Schlüsselpaar** an: Der **private Schlüssel** bleibt bei dir (nie weitergeben, nie ins Repo), der **öffentliche** (Endung `.pub`) kommt zu GitHub.

Schritt 1: Schlüssel erzeugen, Rechte setzen. Im Schulnetz ist `C:\` gesperrt, dort läge aber der Standardordner `~/.ssh`. Lege den Schlüssel deshalb in einen eigenen Ordner auf einem Laufwerk, auf das du schreiben darfst (Netzlaufwerk, USB-Stick) – hier `H:\keys` bzw. `~/keys` (bei euch z. B. `U:\Dokumente\keys`; Pfade in allen Beispielen anpassen), nie im Projektordner. Gib bei der Passphrase-Abfrage unbedingt eine ein.

```
# Linux / macOS
mkdir -p ~/keys
chmod 700 ~/keys                          # Ordner: nur du
ssh-keygen -t ed25519 -C "vorname@schule" -f ~/keys/id_ed25519
chmod 600 ~/keys/id_ed25519               # privat: nur du
chmod 644 ~/keys/id_ed25519.pub           # öffentlich: lesbar
```

```
# Windows (PowerShell)
mkdir H:\keys
ssh-keygen -t ed25519 -C "vorname@schule" -f H:\keys\id_ed25519
icacls H:\keys\id_ed25519 /inheritance:r
icacls H:\keys\id_ed25519 /grant:r "${env:USERNAME}:R"
```

`ssh` verweigert Schlüssel, die andere Konten lesen könnten (`UNPROTECTED PRIVATE KEY FILE, Permissions are too open`). Die `icacls`-Zeilen entfernen geerbte Rechte und erlauben nur deinem Konto das Lesen; `icacls H:\keys\id_ed25519` zeigt zur Kontrolle, wer Zugriff hat.

Schritt 2: Öffentlichen Schlüssel bei GitHub eintragen. Inhalt der `.pub`-Datei anzeigen (`cat ~/keys/id_ed25519.pub` bzw. `Get-Content H:\keys\id_ed25519.pub`), dann auf GitHub: Profilbild → **Settings** → **SSH and GPG keys** → **New SSH key**, Titel vergeben (z. B. „Schule“), Zeile einfügen. Niemals die Datei *ohne* `.pub` hochladen oder verschicken!

Schritt 3: Git den Schlüssel zeigen. Außerhalb von `~/.ssh` findet `ssh` den Schlüssel nicht von selbst. Die Einstellung gilt pro Repo (im Projektordner ausführen) und braucht keinen Zugriff auf die globale Git-Konfiguration. Ein neues Repo klonst du direkt mit der Einstellung; ein vorhandenes stellst du auf die SSH-Adresse um:

```
git config core.sshCommand "ssh -i H:/keys/id_ed25519 -o IdentitiesOnly=yes"
git remote set-url origin git@github.com:dein-user/mein-repo.git

# neues Repo klonen (Einstellung wird gleich mitgespeichert)
git clone -c core.sshCommand="ssh -i H:/keys/id_ed25519 -o IdentitiesOnly=yes"
git@github.com:user/repo.git
```

Schritt 4: Testen (unter Linux Pfad `~/keys/...`):

```
ssh -T -i H:/keys/id_ed25519 -o IdentitiesOnly=yes git@github.com
# Erwartet: "Hi dein-user! You've successfully authenticated, ..."
```

Beim ersten Verbinden fragt SSH, ob dem Server vertraut wird. Vergleiche den Fingerprint mit der GitHub-Dokumentation (`SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU`, ED25519) und antworte nur dann mit `yes`.

Wichtig im Schulalltag

- **Port 22 gesperrt?** GitHub bietet SSH auch über Port 443: `git remote set-url origin ssh://git@ssh.github.com:443/dein-user/mein-repo.git`
- **Gemeinsam genutzte Rechner:** nichts zurücklassen. Schlüssel verloren oder weitergegeben? Auf GitHub unter **SSH and GPG keys** löschen und neu anlegen.

Wenn es hakt: Netzlaufwerk und UNC-Pfade. Ob sich Rechte setzen lassen, entscheidet der Dateiserver, nicht die Schreibweise des Pfads. Nimm trotzdem das gemappte Laufwerk (`U:\Dokumente`), wenn es eines gibt: Ein UNC-Pfad (`\\server\freigabe\...`) bringt zusätzliche Stolperstellen. `cmd` kann ihn nicht als Arbeitsverzeichnis nutzen (stattdessen `pushd \\server\freigabe\ordner` oder PowerShell), und in `core.sshCommand` gehören Schrägstriche hinein: `//server/freigabe/keys/id_ed25519`. Diagnose in drei Befehlen:

```
icacls U:\Dokumente\keys\id_ed25519     # wer hat Zugriff? Nur du!
ssh -v -T -i U:/Dokumente/keys/id_ed25519 git@github.com
Get-Command ssh                           # Windows-ssh oder das von Git?
```

Symptom	Ursache und Lösung
`icacls`: *Zugriff verweigert*	Auf dieser Freigabe darfst du keine Rechte ändern. Anderen Ordner nehmen (am besten einen, den du selbst angelegt hast), gemapptes Laufwerk statt UNC, oder Rückfall (letzte Zeile)
`Permissions ... too open`, obwohl `icacls` lief	In der `icacls`-Ausgabe stehen noch Gruppen (Administratoren, Lehrer, Jeder), oder der Server (Samba/NAS) speichert keine Windows-Rechte. Anderes Laufwerk nehmen oder Rückfall
`dubious ownership in repository`	Das Repo auf der Freigabe gehört nicht deinem Konto. Ausnahme nur in der *globalen* Config möglich (Abschnitt 2, `GIT_CONFIG_GLOBAL`): `git config --global --add safe.directory "U:/Dokumente/projekt"`, bei UNC `"%(prefix)///server/freigabe/projekt"`
Bei jedem Verbinden Frage nach dem Server, oder `known_hosts`-Fehler	Standardordner nicht schreibbar. An die `ssh`-Option anhängen: `-o UserKnownHostsFile=U:/Dokumente/keys/known_hosts`
Schlüssel lässt sich nirgends sicher ablegen	Rückfall ohne Schlüsseldatei: `https://`-Adresse nutzen, beim `push` Benutzername und **Personal Access Token** als Passwort eingeben (GitHub → Settings → Developer settings; Token auf ein Repo und kurze Laufzeit begrenzen). Am Schulrechner nicht dauerhaft speichern lassen

13. Cheatsheet – Schnellreferenz

Aufgabe	Befehl
Repo initialisieren	`git init`
Status prüfen	`git status`
Änderungen stagen	`git add .`
Committen	`git commit -m "msg"`
Hochladen	`git push`
Herunterladen	`git pull`
Repo klonen	`git clone <url>`
Branch erstellen und wechseln	`git switch -c <name>`
Branch wechseln	`git switch <name>`
Branches mergen	`git merge <branch>`
Branch hochladen (PR)	`git push -u origin <branch>`
Änderungen zwischenlagern	`git stash` / `git stash pop`
Verlauf anzeigen	`git log --oneline`
Remote-Verbindung	`git remote add origin <url>`
SSH-Schlüssel erzeugen	`ssh-keygen -t ed25519 -f <pfad>`

14. Typische Fehler und Lösungen

Problem	Lösung
`failed to push some refs`	Erst `git pull`, Konflikte lösen, dann `git push`
Falsche Datei committed	`git rm --cached datei.txt` und in `.gitignore` eintragen
Commit vergessen zu pushen	Einfach `git push` – Git merkt sich offene Commits
`Authentication failed`	GitHub akzeptiert keine Passwörter mehr. SSH-Key einrichten (Abschnitt 12) oder `gh auth login` (GitHub CLI) nutzen. Alternativ: Personal Access Token
`Permission denied (publickey)`	Git findet den Schlüssel nicht oder GitHub kennt ihn nicht: `core.sshCommand` und Pfad prüfen, `.pub`-Schlüssel bei GitHub eintragen (Abschnitt 12)
`Permissions are too open`	Rechte am privaten Schlüssel setzen: `chmod 600` bzw. `icacls` (Abschnitt 12)
Änderungen beim Branchwechsel im Weg	`git stash`, wechseln, später `git stash pop`